

DEVOPS MASTER COURSE

Volume 13A

Platform Engineering Product Foundations

*Product Thinking | Value Streams | Team Topologies | Catalogs | Golden Paths |
Platform APIs*

Chapters 1-10 | 2026 Edition

Beginner to Enterprise Mastery

About This Part

Part A establishes platform engineering as a product and operating discipline. It begins with user research and value streams, then defines team topology, catalogs, golden paths, APIs and an MVP architecture.

2026 curriculum note: The 2026 edition distinguishes an internal developer portal from the broader internal developer platform and uses outcome, adoption and delivery-performance evidence rather than portal page views alone.

Volume 13 Roadmap

Part	Coverage	Status
13A	Platform product foundations and IDP architecture	Current part
13B	Backstage developer portal engineering	Course sequence
13C	Self-service infrastructure and control planes	Course sequence
13D	Golden paths and developer experience	Course sequence
13E	Enterprise platform operations, governance and scale	Course sequence
13F	Projects, interviews and final platform capstone	Course sequence

Scope, Safety and Professional Practice

- Use disposable sandboxes, local simulations or approved training subscriptions. Configure budgets, quotas and expiry before creating resources.
- Never use production identities, customer data, employer account structures, internal addresses or confidential contracts in course artifacts.
- Open-source projects, cloud services, AI models, pricing, limits and certification objectives change. Verify official documentation before production use or exam registration.
- Run failure, security, red-team and recovery tests only in approved nonproduction environments with explicit stop conditions.
- Destroy resources, revoke credentials, remove local state and sanitize every portfolio artifact.

Learning Outcomes

- Explain and design the capabilities covered by Platform Engineering Product Foundations.
- Implement or simulate production workflows through version-controlled automation.
- Test identity, policy, security, reliability, cost and lifecycle behavior.
- Troubleshoot complex failures using a structured evidence-first method.
- Produce sanitized architecture, laboratory, runbook and interview portfolio artifacts.

Table of Contents

- Chapter 1 Platform Engineering as a Product Discipline
- Chapter 2 Value Streams, Cognitive Load and Developer Journeys
- Chapter 3 Platform Team Topology and Interaction Modes
- Chapter 4 Platform Capability Mapping and Maturity
- Chapter 5 Developer Research, Personas and Jobs to Be Done
- Chapter 6 Service Catalog and Ownership Foundations
- Chapter 7 Golden Paths and Paved Roads
- Chapter 8 Platform APIs, Contracts and Versioning
- Chapter 9 Internal Platform Architecture and Control Planes
- Chapter 10 Platform MVP Capstone
- Platform Product and Architecture Reference
- Completion Checklist
- Additional Portfolio Projects
- Practice Plan
- Glossary
- Official Documentation Index

Chapter 1: Platform Engineering as a Product Discipline

Treat the platform as an internal product with users, outcomes, lifecycle ownership and measurable service levels.

Why this chapter matters: Platform Engineering as a Product Discipline matters because enterprise outcomes depend on more than a successful tool installation. The team must make treat the platform as an internal product with users, outcomes, lifecycle ownership and measurable service levels. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of platform engineering as a product discipline.
- Design the capability using CNCF platform guidance, product discovery, service blueprints with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: create a platform product charter and measurable north-star outcomes.

Core Concepts

Treat the platform as an internal product with users and problem framing

Treat the platform as an internal product with users and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For platform engineering as a product discipline, record assumptions and the evidence that would disprove them.

outcomes architecture and operating boundaries

The architecture for outcomes architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

lifecycle ownership and measurable service levels. automation, policy and evidence

Automation for lifecycle ownership and measurable service levels. automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Platform Engineering as a Product Discipline failure handling and continuous improvement

Continuous improvement for platform engineering as a product discipline failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	CNCF platform guidance	Implement the core capability and define its service contract.
Automation and integration	product discovery	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	service blueprints	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	CNCF platform guidance	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	product discovery	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	service blueprints	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: platform_engineering_as_a_product
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 1

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 1: Create a platform product charter and measurable north-star outcomes

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for platform engineering as a product discipline.
2. Create the smallest safe implementation or detailed simulation using CNCF platform guidance, product discovery, service blueprints.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Platform Engineering as a Product Discipline?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Platform Engineering as a Product Discipline is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 2: Value Streams, Cognitive Load and Developer Journeys

Map delivery friction and reduce unnecessary cognitive load without hiding essential operational responsibility.

Why this chapter matters: Value Streams, Cognitive Load and Developer Journeys matters because enterprise outcomes depend on more than a successful tool installation. The team must make map delivery friction and reduce unnecessary cognitive load without hiding essential operational responsibility. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of value streams, cognitive load and developer journeys.
- Design the capability using Value-stream mapping, journey maps, Wardley mapping with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: map one idea-to-production journey and identify platform interventions.

Core Concepts

Map delivery friction and reduce unnecessary cognitive load without hiding essential operational responsibility. and problem framing

Map delivery friction and reduce unnecessary cognitive load without hiding essential operational responsibility. and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For value streams, cognitive load and developer journeys, record assumptions and the evidence that would disprove them.

Value Streams, Cognitive Load and Developer Journeys architecture and operating boundaries

The architecture for value streams, cognitive load and developer journeys architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

Value Streams, Cognitive Load and Developer Journeys automation, policy and evidence

Automation for value streams, cognitive load and developer journeys automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Value Streams, Cognitive Load and Developer Journeys failure handling and continuous improvement

Continuous improvement for value streams, cognitive load and developer journeys failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Value-stream mapping	Implement the core capability and define its service contract.
Automation and integration	journey maps	Connect the capability to version-controlled workflows and APIs.

Capability	Tool or pattern	Enterprise use
Policy, security or governance	Wardley mapping	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Value-stream mapping	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	journey maps	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	Wardley mapping	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: value_streams_cognitive_load_and
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 2

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence

Symptom	Likely cause	Corrective action	Validation evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 2: Map one idea-to-production journey and identify platform interventions

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for value streams, cognitive load and developer journeys.
2. Create the smallest safe implementation or detailed simulation using Value-stream mapping, journey maps, Wardley mapping.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Value Streams, Cognitive Load and Developer Journeys?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Value Streams, Cognitive Load and Developer Journeys is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 3: Platform Team Topology and Interaction Modes

Design platform, stream-aligned, enabling and complicated-subsystem responsibilities with explicit interaction contracts.

Why this chapter matters: Platform Team Topology and Interaction Modes matters because enterprise outcomes depend on more than a successful tool installation. The team must make design platform, stream-aligned, enabling and complicated-subsystem responsibilities with explicit interaction contracts. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of platform team topology and interaction modes.
- Design the capability using Team APIs, RACI, enabling engagements, service ownership with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: design a platform team operating model for ten product teams.

Core Concepts

Design platform and problem framing

Design platform and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For platform team topology and interaction modes, record assumptions and the evidence that would disprove them.

stream-aligned architecture and operating boundaries

The architecture for stream-aligned architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

enabling and complicated-subsystem responsibilities with explicit interaction contracts. automation, policy and evidence

Automation for enabling and complicated-subsystem responsibilities with explicit interaction contracts. automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Platform Team Topology and Interaction Modes failure handling and continuous improvement

Continuous improvement for platform team topology and interaction modes failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Team APIs	Implement the core capability and define its service contract.
Automation and integration	RACI	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	enabling engagements	Constrain behavior with least privilege, validation and exceptions.

Capability	Tool or pattern	Enterprise use
Telemetry and operational evidence	service ownership	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	Team APIs	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	RACI	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: platform_team_topology_and_inter
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 3

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence

Symptom	Likely cause	Corrective action	Validation evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 3: Design a platform team operating model for ten product teams

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for platform team topology and interaction modes.
2. Create the smallest safe implementation or detailed simulation using Team APIs, RACI, enabling engagements, service ownership.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Platform Team Topology and Interaction Modes?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Platform Team Topology and Interaction Modes is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 4: Platform Capability Mapping and Maturity

Assess current capabilities across people, process, policy, technology and business outcomes.

Why this chapter matters: Platform Capability Mapping and Maturity matters because enterprise outcomes depend on more than a successful tool installation. The team must make assess current capabilities across people, process, policy, technology and business outcomes. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of platform capability mapping and maturity.
- Design the capability using CNCF maturity model, capability maps, heatmaps with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: create a baseline maturity assessment and target state.

Core Concepts

Assess current capabilities across people and problem framing

Assess current capabilities across people and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For platform capability mapping and maturity, record assumptions and the evidence that would disprove them.

process architecture and operating boundaries

The architecture for process architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

policy automation, policy and evidence

Automation for policy automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

technology and business outcomes. failure handling and continuous improvement

Continuous improvement for technology and business outcomes. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	CNCF maturity model	Implement the core capability and define its service contract.
Automation and integration	capability maps	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	heatmaps	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	CNCF maturity model	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	capability maps	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	heatmaps	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: platform_capability_mapping_and_
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 4

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 4: Create a baseline maturity assessment and target state

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for platform capability mapping and maturity.
2. Create the smallest safe implementation or detailed simulation using CNCF maturity model, capability maps, heatmaps.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Platform Capability Mapping and Maturity?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Platform Capability Mapping and Maturity is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 5: Developer Research, Personas and Jobs to Be Done

Use interviews and telemetry to understand real developer problems before selecting tools.

Why this chapter matters: Developer Research, Personas and Jobs to Be Done matters because enterprise outcomes depend on more than a successful tool installation. The team must make use interviews and telemetry to understand real developer problems before selecting tools. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of developer research, personas and jobs to be done.
- Design the capability using Interview guides, surveys, support data, JTBD with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: run a fictional developer-research sprint and synthesize findings.

Core Concepts

Use interviews and telemetry to understand real developer problems before selecting tools. and problem framing

Use interviews and telemetry to understand real developer problems before selecting tools. and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For developer research, personas and jobs to be done, record assumptions and the evidence that would disprove them.

Developer Research, Personas and Jobs to Be Done architecture and operating boundaries

The architecture for developer research, personas and jobs to be done architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

Developer Research, Personas and Jobs to Be Done automation, policy and evidence

Automation for developer research, personas and jobs to be done automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Developer Research, Personas and Jobs to Be Done failure handling and continuous improvement

Continuous improvement for developer research, personas and jobs to be done failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Interview guides	Implement the core capability and define its service contract.
Automation and integration	surveys	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	support data	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	JTBD	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	Interview guides	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	surveys	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: developer_research_personas_and_
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 5

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 5: Run a fictional developer-research sprint and synthesize findings

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for developer research, personas and jobs to be done.
2. Create the smallest safe implementation or detailed simulation using Interview guides, surveys, support data, JTBD.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Developer Research, Personas and Jobs to Be Done?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Developer Research, Personas and Jobs to Be Done is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 6: Service Catalog and Ownership Foundations

Establish authoritative service metadata, ownership, lifecycle, dependencies and operational readiness.

Why this chapter matters: Service Catalog and Ownership Foundations matters because enterprise outcomes depend on more than a successful tool installation. The team must make establish authoritative service metadata, ownership, lifecycle, dependencies and operational readiness. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of service catalog and ownership foundations.
- Design the capability using Catalog schemas, metadata validation, ownership policies with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: define a service catalog schema and onboard five sample services.

Core Concepts

Establish authoritative service metadata and problem framing

Establish authoritative service metadata and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For service catalog and ownership foundations, record assumptions and the evidence that would disprove them.

ownership architecture and operating boundaries

The architecture for ownership architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

lifecycle automation, policy and evidence

Automation for lifecycle automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

dependencies and operational readiness. failure handling and continuous improvement

Continuous improvement for dependencies and operational readiness. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Catalog schemas	Implement the core capability and define its service contract.
Automation and integration	metadata validation	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	ownership policies	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Catalog schemas	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	metadata validation	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	ownership policies	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: service_catalog_and_ownership_fo
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 6

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 6: Define a service catalog schema and onboard five sample services

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for service catalog and ownership foundations.
2. Create the smallest safe implementation or detailed simulation using Catalog schemas, metadata validation, ownership policies.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Service Catalog and Ownership Foundations?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Service Catalog and Ownership Foundations is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 7: Golden Paths and Paved Roads

Create opinionated, optional defaults that accelerate common work while preserving escape hatches.

Why this chapter matters: Golden Paths and Paved Roads matters because enterprise outcomes depend on more than a successful tool installation. The team must make create opinionated, optional defaults that accelerate common work while preserving escape hatches. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of golden paths and paved roads.
- Design the capability using Golden-path design, templates, policy, documentation with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: design a web-service golden path with an exception process.

Core Concepts

Create opinionated and problem framing

Create opinionated and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For golden paths and paved roads, record assumptions and the evidence that would disprove them.

optional defaults that accelerate common work while preserving escape hatches. architecture and operating boundaries

The architecture for optional defaults that accelerate common work while preserving escape hatches. architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

Golden Paths and Paved Roads automation, policy and evidence

Automation for golden paths and paved roads automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Golden Paths and Paved Roads failure handling and continuous improvement

Continuous improvement for golden paths and paved roads failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Golden-path design	Implement the core capability and define its service contract.
Automation and integration	templates	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	policy	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	documentation	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	Golden-path design	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	templates	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: golden_paths_and_paved_roads
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 7

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 7: Design a web-service golden path with an exception process

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for golden paths and paved roads.
2. Create the smallest safe implementation or detailed simulation using Golden-path design, templates, policy, documentation.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Golden Paths and Paved Roads?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Golden Paths and Paved Roads is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 8: Platform APIs, Contracts and Versioning

Expose self-service capabilities through stable APIs and contracts rather than ticket queues.

Why this chapter matters: Platform APIs, Contracts and Versioning matters because enterprise outcomes depend on more than a successful tool installation. The team must make expose self-service capabilities through stable apis and contracts rather than ticket queues. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of platform apis, contracts and versioning.
- Design the capability using REST, Kubernetes APIs, schemas, semantic versioning with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: write a versioned platform api contract and compatibility policy.

Core Concepts

Expose self-service capabilities through stable APIs and contracts rather than ticket queues. and problem framing

Expose self-service capabilities through stable APIs and contracts rather than ticket queues. and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For platform apis, contracts and versioning, record assumptions and the evidence that would disprove them.

Platform APIs, Contracts and Versioning architecture and operating boundaries

The architecture for platform apis, contracts and versioning architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

Platform APIs, Contracts and Versioning automation, policy and evidence

Automation for platform apis, contracts and versioning automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

Platform APIs, Contracts and Versioning failure handling and continuous improvement

Continuous improvement for platform apis, contracts and versioning failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	REST	Implement the core capability and define its service contract.
Automation and integration	Kubernetes APIs	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	schemas	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	semantic versioning	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	REST	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	Kubernetes APIs	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: platform_apis_contracts_and_vers
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 8

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 8: Write a versioned platform API contract and compatibility policy

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for platform apis, contracts and versioning.
2. Create the smallest safe implementation or detailed simulation using REST, Kubernetes APIs, schemas, semantic versioning.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Platform APIs, Contracts and Versioning?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Platform APIs, Contracts and Versioning is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 9: Internal Platform Architecture and Control Planes

Separate portal, orchestration, control-plane, execution, policy and telemetry concerns.

Why this chapter matters: Internal Platform Architecture and Control Planes matters because enterprise outcomes depend on more than a successful tool installation. The team must make separate portal, orchestration, control-plane, execution, policy and telemetry concerns. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of internal platform architecture and control planes.
- Design the capability using Portals, workflow engines, Crossplane, Terraform, Kubernetes with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: produce a logical and deployment architecture for an idp.

Core Concepts

Separate portal and problem framing

Separate portal and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For internal platform architecture and control planes, record assumptions and the evidence that would disprove them.

orchestration architecture and operating boundaries

The architecture for orchestration architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

control-plane automation, policy and evidence

Automation for control-plane automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

execution failure handling and continuous improvement

Continuous improvement for execution failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Portals	Implement the core capability and define its service contract.
Automation and integration	workflow engines	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	Crossplane	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	Terraform	Expose health, usage, audit and user-outcome signals.
Lifecycle and recovery	Kubernetes	Define upgrade, rollback, backup, restore and deprecation behavior.

Capability	Tool or pattern	Enterprise use
Portfolio proof	Portals	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: internal_platform_architecture_a
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 9

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 9: Produce a logical and deployment architecture for an IDP

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for internal platform architecture and control planes.
2. Create the smallest safe implementation or detailed simulation using Portals, workflow engines, Crossplane, Terraform, Kubernetes.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Internal Platform Architecture and Control Planes?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Internal Platform Architecture and Control Planes is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Chapter 10: Platform MVP Capstone

Combine discovery, catalog, golden path, API and operating model into an adoptable first platform release.

Why this chapter matters: Platform MVP Capstone matters because enterprise outcomes depend on more than a successful tool installation. The team must make combine discovery, catalog, golden path, api and operating model into an adoptable first platform release. explicit, assign ownership, automate repeatable controls, and retain evidence that the design behaves correctly during normal use, denied use and failure.

Learning Objectives

- Explain the purpose, consumers and enterprise trade-offs of platform mvp capstone.
- Design the capability using Backstage, Git, CI/CD, IaC, observability with explicit ownership and failure boundaries.
- Implement a version-controlled workflow with identity, policy, telemetry and lifecycle controls.
- Troubleshoot common failures and produce evidence suitable for operations, security and architecture review.
- Complete the lab: deliver and defend a platform mvp roadmap and prototype.

Core Concepts

Combine discovery and problem framing

Combine discovery and problem framing starts with a clear user or business problem. Define the desired outcome, the actors, the service boundary and the constraints before selecting implementation technology. For platform mvp capstone, record assumptions and the evidence that would disprove them.

catalog architecture and operating boundaries

The architecture for catalog architecture and operating boundaries must separate control, execution, data and evidence paths. Document trust boundaries, dependencies, quotas, failure domains and who owns each operational decision. Abstraction should reduce repeated work without concealing critical behavior.

golden path automation, policy and evidence

Automation for golden path automation, policy and evidence must be versioned, reviewed and testable. Include schema validation, policy, dry runs or plans, staged promotion, post-change verification and rollback. Manual emergency actions require an owner, timestamp, reason and expiry.

API and operating model into an adoptable first platform release. failure handling and continuous improvement

Continuous improvement for api and operating model into an adoptable first platform release. failure handling and continuous improvement requires observable user outcomes and bounded failure exercises. Test one expected path, one denied path and one dependency failure; then update documentation, alerts and product backlog from the evidence.

Capability and Tool Map

Capability	Tool or pattern	Enterprise use
Primary platform or framework	Backstage	Implement the core capability and define its service contract.
Automation and integration	Git	Connect the capability to version-controlled workflows and APIs.
Policy, security or governance	CI/CD	Constrain behavior with least privilege, validation and exceptions.
Telemetry and operational evidence	IaC	Expose health, usage, audit and user-outcome signals.

Capability	Tool or pattern	Enterprise use
Lifecycle and recovery	observability	Define upgrade, rollback, backup, restore and deprecation behavior.
Portfolio proof	Backstage	Produce sanitized architecture, test and decision evidence.

Architecture and Operating Decisions

- Start with an explicit consumer outcome and service contract; do not expose a tool merely because it is available.
- Separate control-plane authority from workload execution and minimize the maximum blast radius of credentials, state and change.
- Prefer short-lived identity, immutable artifacts, versioned configuration and automated policy over persistent credentials and console-only workflows.
- Define SLOs, ownership, support, cost allocation, recovery, upgrade and deprecation before broad adoption.
- Preserve an escape hatch with accountable review when the golden path cannot meet a valid requirement.

Reference Service Contract

```

capability: platform_mvp_capstone
owner: platform-or-product-team
consumers: [developer, operator, security, finance]
contract:
  inputs: [request, metadata, policy_context]
  outputs: [resource_or_service, status, evidence]
controls:
  identity: short_lived_and_least_privilege
  policy: versioned_and_tested
  telemetry: metrics_logs_traces_audit
  lifecycle: provision_upgrade_recover_deprecate
validation:
  - approved_path
  - denied_path
  - dependency_failure
  - rollback_or_restore
  - cost_and_ownership_check
volume: 13A
chapter: 10

```

Production Readiness Checklist

- [] Consumers, owner, support tier, lifecycle state and service-level objectives are recorded.
- [] Identity, tenant boundaries, data classification, network paths and secrets are tested.
- [] Configuration and policy are versioned, reviewed, promoted and observable.
- [] Failure, rollback or restore, capacity, quota and dependency behavior have been exercised.
- [] Usage, adoption, user outcomes, cost allocation and exception expiry are measurable.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Request remains pending	Controller, runner or dependency cannot reconcile the desired state	Inspect status conditions, events, queue depth and dependency health; identify the first failing boundary	Successful reconciliation plus latency and audit evidence
User is denied unexpectedly	Identity mapping, policy context or ownership metadata is incomplete	Evaluate effective identity and policy inputs; correct the narrowest cause and rerun denied/allowed tests	Expected allow and deny results with access logs
Change succeeds but user outcome degrades	Technical success criteria do not represent service behavior	Compare SLO, dependency and user-journey signals before and after the change; roll back if needed	Restored SLO and documented causal evidence
Cost or capacity grows unexpectedly	Unbounded defaults, leaks, retries, idle resources or poor allocation	Break down consumption by tenant and unit; apply quota, TTL, scaling or design changes	Reduced unit cost with reliability and quality unchanged

Hands-on Lab 10: Deliver and defend a platform MVP roadmap and prototype

Prerequisites

- Disposable local or cloud sandbox with budgets and expiry controls.
- Git, containers, Kubernetes or a documented simulation as appropriate.
- Synthetic identities and data only; do not use employer or customer secrets.

Implementation

1. Define fictional requirements, users, ownership and acceptance criteria for platform mvp capstone.
2. Create the smallest safe implementation or detailed simulation using Backstage, Git, CI/CD, IaC, observability.
3. Add identity, policy, metadata, telemetry, quota, cost and lifecycle controls.
4. Run one successful user journey and one intentionally denied action.
5. Inject a bounded dependency or configuration failure, diagnose it, recover and record the timeline.
6. Publish a sanitized architecture, runbook, evidence pack and improvement backlog; then clean up all resources.

Validation Evidence

- Architecture and service contract committed to version control.
- Commands, test results, screenshots or exported telemetry for allow, deny and failure scenarios.
- Recovery, cost, ownership, security and cleanup evidence with timestamps.

Interview Questions and Model Answers

1. How would you design Platform MVP Capstone?

Begin with consumers, outcomes, constraints and ownership. Define the service contract and failure boundaries, choose the minimum viable architecture, automate policy and lifecycle, then prove normal, denied and recovery behavior.

2. Which trade-offs would you document?

Document speed versus control, abstraction versus transparency, standardization versus flexibility, managed service versus portability, centralization versus team autonomy, and cost versus resilience.

3. What production evidence is required?

Require version-controlled architecture and configuration, effective-access results, policy tests, user-journey telemetry, SLOs, rollback or restore evidence, ownership, cost allocation and a clean teardown or deprecation path.

4. How would you improve adoption?

Use developer research, remove the largest friction, ship a narrow golden path, publish support and escape hatches, measure time-to-value and satisfaction, and iterate with representative product teams.

Chapter Summary

- Platform MVP Capstone is an owned product or operating capability, not a one-time installation.
- Enterprise readiness requires explicit contracts, boundaries, policy, telemetry and lifecycle controls.
- Adoption and reliability improve when normal, denied and failure behavior are tested with users.
- The lab artifact should show reasoning and evidence, not confidential environment details.

Platform Product and Architecture Reference

Use this reference to review product outcomes, architecture boundaries, operating controls and evidence required for production approval.

Domain	Required outcome	Validation evidence
Product and users	Named consumers, outcome hypothesis, roadmap and lifecycle owner	Research summary, service contract and adoption data
Architecture	Control, execution, data and evidence boundaries with bounded blast radius	Diagrams, ADRs, dependency and failure tests
Identity and security	Least privilege, keyless workloads, protected administration and auditable exceptions	Effective-access, denied-action and audit evidence
Delivery and policy	Versioned automation, policy, promotion, rollback and provenance	Pipeline, policy and artifact verification results
Reliability and operations	SLOs, telemetry, capacity, incident response, recovery and deprecation	Dashboards, game-day timeline and restore evidence
Data and AI governance	Ownership, classification, quality, privacy, lineage and responsible use	Catalog, tests, risk and approval records
Economics and value	Allocated cost, unit economics, investment rationale and optimization safety	Cost model, outcome metrics and decision log
People and governance	Clear decisions, support, skills, change plan and feedback loops	RACI, operating cadence, survey and action backlog

Completion Checklist

- [] I can explain every chapter in Volume 13A using architecture, operations and business language.
- [] I can implement or simulate at least one complete service with identity, policy, telemetry and lifecycle controls.
- [] I can demonstrate expected, denied and failure behavior with time-stamped evidence.
- [] I can discuss security, reliability, cost, adoption and organizational trade-offs.
- [] I can produce a sanitized portfolio artifact and defend the design in an interview or review board.

Additional Portfolio Projects

1. Service catalog and ownership quality automation.
2. Keyless CI/CD and workload-identity reference implementation.
3. Policy promotion pipeline with expiring exceptions.
4. Golden path or pipeline template with user-journey telemetry.
5. SLO dashboard and bounded failure game day.
6. Cost allocation and unit-economics review.
7. Security threat model and evidence-based remediation plan.
8. Backup, restore, failover and failback exercise.
9. Migration or adoption factory for three fictional teams.
10. Executive architecture or investment memo with measured outcomes.

Practice Plan

Days	Focus	Required output
1-5	Concepts, users and current-state discovery	Research notes, value stream and capability map
6-10	Architecture and service contracts	Diagrams, API/schema and ADRs
11-16	Implementation and automation	Working sandbox or detailed simulation
17-21	Security, policy and governance	Threat model, policy tests and exceptions
22-26	Reliability, telemetry and recovery	SLOs, dashboards and failure drill
27-31	Cost, capacity and lifecycle	Unit cost, quota and deprecation plan
32-36	Adoption, people and operating model	RACI, support and change plan
37-41	Troubleshooting and interview practice	Timed investigation and interview notes
42-45	Capstone and portfolio	Final defense, cleanup and sanitized evidence pack

Glossary

Term	Definition
Control plane	The APIs and controllers that accept desired state and coordinate execution.
Developer portal	A user interface that aggregates catalog, documentation, workflows and operational context.
Internal developer platform	The integrated products, APIs, automation and processes that support software delivery.
Golden path	An opinionated, supported default journey for a common engineering task.
Data product	An owned data asset or interface with consumers, quality, documentation and lifecycle.
MLOps	Practices and systems for reproducible, governed delivery and operation of machine-learning models.
LLMOps	Lifecycle, evaluation, deployment, observability, safety and cost practices for LLM applications.
Sociotechnical system	A system whose outcomes emerge from interactions among people, process, incentives and technology.
Unit economics	Cost and value measured per meaningful business or technical output.
Error budget	The allowed unreliability implied by a service-level objective.
Policy as code	Machine-testable policy stored, reviewed and promoted through version control.
Evidence pack	Sanitized architecture, configuration, test, telemetry and decision records used to prove readiness.

Official Documentation Index

The following primary or official sources were consulted for this 2026 edition. Verify current versions, service behavior, prices, limits and certification objectives before use.

- [Backstage - What is Backstage?](#)
- [Backstage Software Catalog](#)
- [Backstage Software Templates](#)
- [Backstage TechDocs](#)

- [Crossplane Documentation](#)
- [Crossplane Compositions](#)
- [CNCF Platforms White Paper](#)
- [DORA Platform Engineering Capability](#)
- [Kubernetes Documentation](#)
- [OpenGitOps Principles](#)

Volume 13A completion: preserve your architecture, code or simulation, test evidence, recovery timeline, cost/value review and sanitized final report as one portfolio package.